

SZEGEDI TUDOMANYEGYETEM

Informatikai Intezet

SZAKDOLGOZAT

TDLWebshop - épületgépeszeti webshop és adminisztrációs rendszer

Toth David Laszlo

Szeged, 2026

TDLWebshop - épületgépészeti webshop és adminisztrációs rendszer

Mezo	Tartalom
Készítette	Toth David Laszlo
Szak	[pontos szak megadása]
Intezet / tanszek	[intezményi követelmény szerinti pontos megnevezés]
Temavezető	Dr. Bilicki Vilmos, egyetemi docens
Hely és év	Szeged, 2026

Megjegyzés: a szögletes zárójelekben szereplő adatokat a végleges formai követelmények alapján kell kitölteni.

Feladatkiiras

A szakdolgozat feladata egy epulelgepeszeti termekeket kezelo webaruhaz es a hozza kapcsolodo adminisztracios rendszer megtervezese es megvalositasa. A rendszer celja, hogy egy kisebb vagy kozepes meretu szakmai kereskedes digitalis mukodeset tamogassa: a vesarloi oldalon termekbongeszes, kosar, rendelesleadas, profil es rendelestortenet erhető el, mig az admin oldalon termek-, keszlet-, rendeles-, felhasznalo- es helyszini vasarlaskezeles jelenik meg.

A munka soran kiemelt szempont volt, hogy a webshop ne csak latvanyos felulet legyen, hanem mernokileg is ertelmezhető MVP-kent mukodjon. Ennek resze a jogosultsagi modell, a Firestore biztonsagi szabalyok, a tranzakcios rendeleskezeles, a PDF bizonylat/szamla generalasa, a CSV alapu termekimport, valamint egy katalogushoz kotott AI asszisztens prototipusa is. A feladat hatara tudatosan nem egy teljes erteku, bankkartya-elfogadassal es vallalati ERP-integracioval rendelkezo eles webshop, hanem egy bizonyithatoan mukodo, szakdolgozati szinten bemutatathato webes rendszer.

Tartalmi összefoglaló

Szempont	Leiras
Tema megnevezese	TDLWebshop - epulelgepeszeti webshop es adminisztracios rendszer
Feladat	Egy olyan webes alkalmazas letrehozasa, amely vesarloi es adminisztracios folyamatokat is kezel.
Megoldasi mod	Angular alapu frontend, Firebase/Firestore adattarolas es jogosultsagkezeles, Cloudflare Worker alapu AI proxy.
Alkalmazott eszkozok	Angular, TypeScript, Firebase Authentication, Firestore, Firebase Hosting, Cloudflare Workers, OpenRouter API, GitHub Actions.
Elert eredmenyek	Mukodo vesarloi ut, admin felulet, rendeleskezeles, keszletfigyeles, CSV import, PDF bizonylat, AI asszisztens es dokumentalt teszteles.
Kulcsszavak	webshop, Angular, Firebase, Firestore, admin rendszer, epulelgepeszet, AI asszisztens, CI, MVP

Tartalomjegyzék

A végleges dokumentumban ezt Word automatikus tartalomjegyzekkel kell frissíteni.

Javaslat: Hivatkozások -> Tartalomjegyzék -> Automatikus tartalomjegyzék, majd teljes frissítés.

1. Bevezetes, problemafelvetes es celkituzes

Az online kereskedelemben a felhasznalok ma mar nem csak egy egyszeru termeklistat varnak el, hanem gyors keresest, egyertelmu keszletinformaciot, megbizhato rendelési folyamatot es atlathato kommunikaciót. Ez különösen igaz az épütelgépészeti területre, ahol a termékek gyakran szakmai döntést igényelnek, a vásárló pedig nem mindig tudja pontosan, melyik alkatrész vagy berendezés illik a saját helyzetéhez.

A TDLWebshop alapötlete ebből a problémából indult ki. A cél egy olyan webáruház volt, amely nem általános fogyasztási cikkekre, hanem fűtéshez, vízszereleshez, hűtőtechnikához, szellőzéshez és szerelési anyagokhoz kapcsolódó termékekre épül. A rendszer egyszerre célozza a lakossági vásárlókat és azokat a szakmai vagy üzleti szereplőket, akik gyakran helyszíni vagy B2B jellegű vásárlást igényelnek.

A szakdolgozat nem csupán a frontend megjelenítésre összpontosít, hanem a teljesebb folyamatra: a termékadatok kezelésére, a kosár és checkout logikájára, a rendelések követésére, az adminisztrációs munkafolyamatokra, a jogosultságokra, a biztonsági szabályokra és a tesztelhetőségre. A dolgozat célja az is, hogy bemutassa, hogyan lehet egy hallgatói projektet termékszerű MVP irányába vinni.

A dolgozatban bemutatott rendszer fejlesztésénél fontos szempont volt a reprodukálhatóság. A repo tartalmazza a futtatási leírást, a környezeti változók mintáját, a CI ellenőrzést és a tesztelési dokumentációt. Ez azért lényeges, mert a szakdolgozat értékelése során nem elég az, hogy a rendszer egyszer már működött: bizonyítani kell, hogy más környezetben is értelmezhetően elindítható és ellenőrizhető.

1.1. A megoldando problema

Az épütelgépészeti termékek kínálata sokféle kategóriát érint, és a termékek közötti különbség gyakran nem csak árban vagy márkában jelenik meg. Egy radiator, csaptelep, szivattyú, klíma vagy szellőztető esetében fontos lehet a méret, teljesítmény, alkalmazási terület, kompatibilitás és készlet. Emiatt a felhasználó számára különösen értékes, ha az oldal nem csak listáz, hanem segíti a választást.

A másik probléma az adminisztráció. Egy kisebb kereskedésnél a webes rendelések mellett előfordulhat helyszíni vásárlás, telefonos egyeztetés, mentett vásárló, céges kedvezmény, készletfigyelés és PDF bizonylat igény is. Ha ezek külön rendszerekben vagy kézzel vezetett táblázatokban jelennek meg, az hibát és adateltoreést okozhat.

A TDLWebshop ezeket a problémákat egy központi rendszerben kezeli. A vásárlói oldal a termékek megtalálására és a rendelés leadása felől közelít, az admin felület pedig a termékek, rendelések, készlet és vásárlók kezelésére. A két oldal közös adatmodellre épül, így ugyanaz a termék- és rendelésadat jelenik meg a különböző szerepkörök számára.

1.2. Celkituzes es MVP-hatar

A projekt célja nem egy teljes ipari webshoprendszer lemasolása volt, hanem egy olyan MVP létrehozása, amely a legfontosabb vásárlói és adminisztrációs folyamatokat bizonyíthatóan lefedi. Az MVP-határ meghatározása tudatos döntés volt, mert egy szakdolgozati projektben a túl sok felig kész funkció gyengítheti a rendszer megítélését.

1. táblázat - Az MVP-határ összefoglalása

MVP része	Rövid indoklás	Allapot
Terméklista és termékadatlap	A webshop alapja, a vásárlói tájékozódás kiindulópontja.	Megvalósítva
Kosár és checkout	A vásárlói út legfontosabb üzleti folyamata.	Megvalósítva
Rendeléskövetés profilból	Bizalmat és áttekinthetőséget ad a vásárlónak.	Megvalósítva
Admin termék- és készletkezelés	A bolt működtetéséhez szükséges belső folyamat.	Megvalósítva
Helyszíni vásárlás	Domainhez illeszkedő B2B/üzleti különbség.	Megvalósítva
PDF bizonylat/számla	Adminisztrációs bizonyíték és szakdolgozati érték.	Megvalósítva
AI asszisztens	Katalógushoz kötött, korlátozott szakmai segítség.	Prototípus
Bankkártya-elfogadás	Valós pénzügyi integráció magasabb kockázattal.	Továbbfejlesztés
ERP integráció	Vállalati szintű illesztés, külső rendszerekkel.	Továbbfejlesztés

[KEP / ABRA HELYE] A TDLWebshop kezdőlapja dark modban

Beillesztendo tartalom: Keszits kepernyokepet a fooldalrol ugy, hogy a kategoriak legordulo menuje nyitva van.

2. Piaci es teruleti osszehasonlitas

A TDLWebshop pozicionalasahoz erdemes megvizsgalni, hogy egy epulelgepeszeti vagy muszaki webshop milyen funkciokat kinal a gyakorlatban. A piaci osszehasonlitas celja nem az, hogy a projekt mindenben felulmulja a letezo rendszereket, hanem hogy megmutassa: a valasztott funkciok valos teruleti igenyekhez kapcsolodnak.

Az altalanos webshopok tobbnyire eros termeklistaval, kategoriakkal, keresovel es kosarral rendelkeznek, de a belso adminisztracios folyamatok vagy nem lathatok, vagy kulon vallalati rendszerben mukodnek. A szakdolgozat szempontjabol a TDLWebshop erteke eppen abban jelenik meg, hogy a vasarloi oldal mellett az admin oldal es a helyszini vasarlas is bemutatott resze a rendszernek.

A domain kulonlegessege, hogy az epulelgepeszeti termekeknek a vasarlo gyakran nem csak gyorsan rendelni akar, hanem szakmai dontesben is segitseget var. Emiatt a katalogushoz kotott AI asszisztens jo szakdolgozati kiegészites, de csak akkor, ha a korlatai vilagosak: nem helyettesiti a szakembert, es nem talalhat ki nem letezo termeket.

2. tablazat - Piaci funkciok osszehasonlitas

Szepont	Altalanos webshop	Epulelgepeszeti webshop	TDLWebshop megoldasa
Termeklista	Altalanos kategoriak es keresesi lehetoseg.	Szakmai kategoriak, meretek, keszlet.	Futes, viz, hutes, szellozes, szerelési anyagok, lakossagi megoldasok.
Admin folyamat	Tobbnyire nem lathato a felhasznalónak.	Keszlet es rendeles gyors kezelese fontos.	Admin felulet, CSV import, keszletfigyeles, helyszini vasarlas.
Rendelesekoveles	Alap statuszok.	Fontos a keszlet es teljesites kovetese.	Profilbol lathato rendelesek es admin statuszvaltas.
B2B jelleg	Gyakran kulon rendszer.	Ceges kedvezmeny es mentett vasarlo fontos.	Mentett vasarlok, ceges jeloles, helyszini rogzitese.

AI támogatás	Ritkán katalógushoz kötött.	Szakmai kérdéseknél hasznos lehet.	Katalógus-alapu, korlátozott AI asszisztens OpenRouter proxyn keresztül.
--------------	-----------------------------	------------------------------------	--

2.1. A saját rendszer értéke

A TDLWebshop fő értéke nem egyetlen különleges funkcióban, hanem a folyamatok együttműködésében van. A vásárló termeket keres, kosárba tesz, rendelést ad le, majd a profiljában követi az állapotot. Az admin ugyanebben az adatkorben dolgozik: módosítja a rendelést, figyeli a készletet, PDF-et generál, illetve helyszíni vásárlást rögzít.

A rendszer oktatási szempontból azért is érdekes, mert több, egymással összefüggő technikai területet mutat be. Ilyen a kliensoldali Angular alkalmazás, a Firestore alapú adattárolás, a Firebase Authentication, a biztonsági szabályok, a CI ellenőrzés, valamint a szerveroldali titokkezeeléssel kialakított AI proxy.

A dolgozatban ezt az értéket úgy kell bizonyítani, hogy nem csak a kész képernyők látszanak, hanem az is, hogyan jut el az adat a felhasználói művelettől a Firestore dokumentumig, hogyan változik a rendelestörténet, és milyen ellenőrzések védik az adatokat.

[KEP / ABRA HELYE] Piaci összehasonlító táblázat

Beillesztendő tartalom: Ide beilleszthető a saját piackutatásból készült rövid táblázat vagy diagram.

3. Követelmények és use case-ek

A követelmények megfogalmazása azért fontos, mert a szakdolgozatban a rendszer működését nem elég felsorolni. Meg kell mutatni, hogy az egyes funkciók milyen felhasználói vagy adminisztrációs igényre adnak választ, és hogyan ellenőrizhető, hogy ezek ténylegesen működnek.

A TDLWebshop követelményei három nagy csoportba sorolhatók. Az első csoport a vásárlói folyamatokat tartalmazza: termékkeresés, kosár, checkout, profil és rendeléskövetés. A második az adminisztrációs folyamatokat fedi le: termékfeltöltés, rendeléskezelés, készlet, vásárló kezelés, kupon és PDF. A harmadik csoport a technikai és

minosegi kovetelmenyeket rogziti, peldaul a jogosultsagokat, biztonsagot, tesztelhetoseget es reprodukaltat.

3. tablazat - Kovetelmeny-traceability osszefoglalo

Azonosito	Kovetelmeny	Use case	Modul / bizonyitek
K1	A felhasznalo tudjon termeket keresni es kategoriak szerint bongeszni.	Termekkek bongeszese	Termeklista oldal, keresesi mezo, kategoriak
K2	A felhasznalo tudjon termeket kosarba tenni es mennyiseget modositani.	Kosar kezeles	CartService, kosar oldal
K3	A checkout ellenorizze az emailt, telefonszamot es kotelezo adatokat.	Rendeles leadasa	checkout.ts validacios logika
K4	A rendeles létrejotte utan az admin lassa es modositani tudja az allapotot.	Admin rendeleskezeles	admin.ts, order.service.ts
K5	A statuszvaltas audit es keszletvaltozas mellett tortenjen.	Rendeles teljesitese	OrderService tranzakcios logika
K6	A dolgozo csak korlatozott admin funkciokat erjen el.	Dolgozoi felulet	Firestore rules, admin jogosultsagi logika
K7	A PDF bizonylat tartalmazza a rendelest, vevot, tetelek es osszegeket.	Szamla / bizonylat letoltese	invoice.service.ts
K8	Az AI asszisztens ne talaljon ki termeket, csak katalogushoz kototten ajanljon.	AI kerdes megvalaszolasa	chatbot-llm.service.ts, Worker proxy

3.1. Fo use case-ek

4. tablázat - Use case-ek rovid összefoglalása

Use case	Nev	Leiras
UC1	Vasarlo termeket keres es kosarba teszi	A vasarlo kategoriat választ vagy keresoszot ad meg, megnyitja a termeket, majd kosarba helyezi.
UC2	Vasarlo rendelest ad le	A vasarlo kitölti a checkout urlapot, a rendszer validál, majd létrehozza a rendelest.
UC3	Admin teljesíti a rendelest	Az admin megnyitja a rendelest, ellenorzi az adatokat, statuszt modosit, a rendszer auditot es keszletvaltozast rogzi.
UC4	Admin helyszini vasarlast rogzi	Az admin mentett vasarlot választ vagy új adatot ad meg, tetelet ad a vasarlashoz, majd PDF bizonylatot general.
UC5	Dolgozo termeket kezel	A dolgozo a számára engedelyezett admin funkciókkal termeket tolthat fel es keszletet ellenorizhet.
UC6	Felhasznalo AI asszisztent használ	A felhasznalo katalogushoz vagy epulelgepeszethez kapcsolodo kerdest tesz fel, a rendszer korlatos választ ad.

3.1.1. Vasarlo termeket keres es kosarba teszi

A vasarlo kategoriat választ vagy keresoszot ad meg, megnyitja a termeket, majd kosarba helyezi.

Sikeres lefutas eseten a rendszer egyertelmu visszajelzest ad, az adat pedig a megfelelo Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag eseten a felhasznalo validacios vagy jogosultsagi uzenetet kap, nem pedig csendes hibaval talalkozik.

3.1.2. Vasarló rendelest ad le

A vasarló kitölti a checkout urlapot, a rendszer validál, majd létrehozza a rendelest.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozík.

3.1.3. Admin teljesíti a rendelest

Az admin megnyitja a rendelest, ellenorzi az adatokat, státuszt modosit, a rendszer auditot és készletváltozást rögzít.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozík.

3.1.4. Admin helyszini vasarlast rögzít

Az admin mentett vasarlot választ vagy új adatot ad meg, tetelet ad a vasarlashoz, majd PDF bizonylatot generál.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozík.

3.1.5. Dolgozó termeket kezel

A dolgozó a számára engedélyezett admin funkciókkal termeket tolthet fel és készletet ellenorizhet.

Sikeres lefutas esetén a rendszer egyértelmu visszajelzést ad, az adat pedig a megfelelő Firestore gyujtemenyben vagy kliensoldali állapotban megjelenik. Hibaag esetén a felhasználó validációs vagy jogosultsági üzenetet kap, nem pedig csendes hibával találkozík.

3.1.6. Felhasználó AI asszisztent használ

A felhasználó katalogushoz vagy épüelgépészethez kapcsolódó kérdést tesz fel, a rendszer korlatos választ ad.

Sikeres lefutas eseten a rendszer egyertelmu visszajelzest ad, az adat pedig a megfelelo Firestore gyujtemenyben vagy kliensoldali allapotban megjelenik. Hibaag eseten a felhasznalo validacios vagy jogosultsagi uzenetet kap, nem pedig csendes hibaval talalkozik.

[KEP / ABRA HELYE] Use case diagram

Beillesztendo tartalom: Ide illesztheto a vasarlo, admin, dolgozo es AI asszisztens kapcsolatát bemutato use case abra.

4. GUI/UX tervezes

A felulet tervezesenel a cel egy olyan modern, sotet alapu, ipari hangulatu arculat kialakitasa volt, amely illeszkedik az epulelgepeszeti temahoz. A dark mode alapertelmezett megjelenes, mert a TDLWebshop logója es a technikai jellegu termekkor jól mukodik ebben a vizualis környezetben. A light mode ugyanakkor fontos kenyelmi es üzleti alternativa.

A design nem pusztan dekorativ szerepet tolt be. A felhasználói folyamatokban a keresosav, kategoriak, kosar, profil, termekkartyak es CTA gombok elhelyezese azt szolgálja, hogy a vasarlo gyorsan eljusson a termekhez es a rendeleshez. Az admin felulet mas logikat követ: ott a surubb, tablazatosabb es funkciokozpontu elrendezes a hatekonyabb.

A GUI/UX fejezetben a vegleges dolgozatba tenyleges kepnyokepeket kell beszurni. Ezek nem csak illusztraciok, hanem bizonyitekok arra, hogy a rendszer nem egy izolalt kodreszlet, hanem hasznalhato felhasználói felulettel rendelkezo webalkalmazas.

5. tablatzat - GUI/UX kepnyok es bizonyitekok

Kepernyo	Cel	Fontos allapot	Bizonyitek
Kezdolap	Első benyomas, kategoria es akcio kiemeles.	Dark mode, kategoriak dropdown.	Kepernyokep
Termeklista	Bongeszes, kereses, szures.	Tobb termek, szurok, akciós termek.	Kepernyokep
Termekadatlap	Reszletes termekadatok es kosarba helyezes.	Kepgaleria, ar, keszlet.	Kepernyokep

Kosar	Mennyiség, összeg, tovabblepes.	Tobb termék, törles es mennyiség.	Kepernyokep
Checkout	Adatbekeres es validacio.	Hibas email/telefon, sikeres rendeles.	Kepernyokep
Profil	Rendelestortenet es adatok.	Korabbi rendelesek, statusz.	Kepernyokep
Admin	Belso folyamatok kezelese.	Termekimport, rendelesek, keszlet.	Kepernyokep
AI asszisztens	Katalogushoz kotott segitseg.	Domain kerdes es valasz.	Kepernyokep

[KEP / ABRA HELYE] Kezdolap dark mode, kategoriak legordulo menuvel

Beillesztendo tartalom: Fooldal, kategoriak menu nyitva.

[KEP / ABRA HELYE] Kezdolap AI asszisztenssel

Beillesztendo tartalom: Fooldal, AI ablak nyitva, egy relevans epulelgepeszeti kerdessel.

[KEP / ABRA HELYE] Termeklista szures/kereses kozben

Beillesztendo tartalom: Termekek oldal, keresoszo vagy categoria szuro aktiv.

[KEP / ABRA HELYE] Termekadatlap

Beillesztendo tartalom: Egy konkret termék oldala, termékkep, ar, keszlet, kosar gomb lathato.

[KEP / ABRA HELYE] Kosar tobb termékkel

Beillesztendo tartalom: Kosar oldal legalabb ket tetellel.

[KEP / ABRA HELYE] Checkout validacio

Beillesztendo tartalom: Hibas email vagy telefonszam peldaval.

[KEP / ABRA HELYE] Checkout sikeres rendeles

Beillesztendo tartalom: Sikeres leadas utani visszajelzes.

[KEP / ABRA HELYE] Profil es rendeleskovetes

Beillesztendo tartalom: Vasarloi profil oldal korabbi rendelessel.

[KEP / ABRA HELYE] Kivansaglista

Beillesztendo tartalom: Kivansaglista oldal mentett termekkel.

[KEP / ABRA HELYE] Admin attekintes

Beillesztendo tartalom: Admin fo nezet statisztikai kartyakkal.

[KEP / ABRA HELYE] Admin termekkezeles es CSV import

Beillesztendo tartalom: Admin termekek ful CSV import resszel.

[KEP / ABRA HELYE] Admin keszletfigyeles

Beillesztendo tartalom: Keszlet ful alacsony keszlet jelzessel.

[KEP / ABRA HELYE] Helyszini vasarlas

Beillesztendo tartalom: Mentett vasarlo kivalasztva, tetelek hozzaadva.

[KEP / ABRA HELYE] PDF szamla vagy bizonylat

Beillesztendo tartalom: General PDF megnyitva, fejlec es tetelsorok lathatok.

[KEP / ABRA HELYE] Admin felhasznalo es jogosultsag kezeles

Beillesztendo tartalom: Felhasznalok ful admin/dolgozo szerepkorokkal.

[KEP / ABRA HELYE] GitHub Actions zold CI

Beillesztendo tartalom: GitHub Actions lista legfrissebb zold futassal.

5. Technologiai hatter

A TDLWebshop Angular alapu egyoldalas webalkalmazaskent keszult. Az Angular valasztasa azert indokolt, mert komponensalapu felépítést, eros TypeScript tamogatast es strukturalt szolgáltatásreteget ad. Egy webshop es admin rendszer esetenben ez kulonosen hasznos, mert a termeklista, kosar, checkout, profil es admin felulet kulon komponensekre bonthato.

Az adattarolasi es hitelesitesi reteget Firebase szolgáltatások adjak. A Firestore dokumentumalapú modellje jól illeszkedik a termek, rendelések, felhasználói profilok, kuponok es auditbejegyzések tarolasahoz. A Firebase Authentication segitsegevel a bejelentkezés es szerepkorhoz kötött felületkezelés is egyszerubben megvalosithato.

A rendszerben megjelenő AI asszisztens külső LLM-szolgáltatásra épül, de a kliens nem közvetlenül hívja az OpenRouter API-t. A kulcs védelme érdekében egy Cloudflare

Worker proxy kezeli a szerveroldali hivasokat. Ez fontos biztonsagi dontes, mert API kulcsot nem szabad nyilvános frontend kodba egetni.

6. tablázat - A rendszerben hasznalt technologiak

Technologia	Szerep a rendszerben	Indoklas
Angular	Frontend keretrendszer	Komponensalapu, TypeScript alapu, nagyobb felulethez jól strukturalt.
TypeScript	Alkalmazaslogika	Tipusossag, attekinthetobb szolgáltatások és komponensek.
Firebase Authentication	Felhasználóazonositas	Emailés belepés, szerepkorhoz kotheto profilok.
Cloud Firestore	Adattarolas	Dokumentumalapú modell, realis webshop entitasokhoz illesztheto.
Firestore Rules	Biztonsagi retegek	Jogosultságok és payload validacio adatbázis szinten.
Firebase Hosting	Publikalas	Egyszeru deploy és webes elérés.
Cloudflare Worker	AI proxy	API kulcs vedelme és CORS kezeles.
GitHub Actions	CI ellenorzes	Build és teszt futtatás bizonyitása.

5.1. Alternativak és döntések

A backend megvalositása tortenhetett volna saját Node.js/Express szerverrel is. A Firebase választása azért volt kedvező, mert a szakdolgozati celhoz gyorsabban adott hitelesitest, hostingot, adatbázist és biztonsagi szabalyokat. Ez nem jelenti azt, hogy minden üzleti logika idealisan kliensoldalon maradhat; a dolgozatban ezt MVP-korlatként is rögzíteni kell.

A bankkartya-elfogadás és teljes penzugyi integracio tudatosan kimaradt. Ennek oka, hogy egy valós fizetési szolgáltató bekotheses jogi, biztonsagi és tesztelesi követelményeket hozna be. A rendszerben a fizetési modok adminisztrativan jelennek meg, de a valós penzmozgást a projekt nem kezeli.

Az AI asszisztens eseteiben szinten fontos volt a határhúzás. A cél nem egy teljes szakmai tanácsadó rendszer, hanem egy katalogushoz kötött, segítő jellegű funkció. Ha a kérdés túlválik vagy nincs pontos terméktalálat, a rendszernek nem szabad magabiztosan kitalált terméket ajánlania.

6. Architektúra és adatáramlás

A rendszer architektúrája három fő rétegre bontható. Az első a kliensoldali Angular alkalmazás, amely a felhasználói felületet, a komponenseket és a szolgáltatásokat tartalmazza. A második a Firebase rétege, ahol a hitelesítés, Firestore adatbázis, biztonsági szabályok és hosting jelenik meg. A harmadik a külső integrációs réteg, amelybe az AI proxy és az OpenRouter szolgáltatás tartozik.

Az Angular alkalmazásban a komponensek a megjelenítésért és felhasználói interakcióért felelnek, míg a szolgáltatások kezelik az adatműveleteket. Ilyen szolgáltatás a kosár, rendelés, számla, profil, AI asszisztens vagy termékkezelés logikája. Ez a szétválasztás segít abban, hogy a felület ne közvetlenül tartalmazzon az összes üzleti logikát.

Az adatáramlás példája a checkout folyamaton jól bemutatható. A vásárló a felületen megadja az adatokat, a komponens validál, majd a rendelési szolgáltatás létrehozza a Firestore dokumentumot. Az admin felület később ugyanezt a rendelest olvassa, státuszt módosít, auditbejegyzést készít és bizonyos esetekben készletváltozást hajt végre.

[KEP / ABRA HELYE] Komponens-architektúra ábra

Beillesztendő tartalom: Angular frontend, Firebase/Firestore, Firestore Rules, Cloudflare Worker és OpenRouter kapcsolatainak bemutatása.

[KEP / ABRA HELYE] Checkout szekvencia ábra

Beillesztendő tartalom: Vásárló -> Angular checkout -> OrderService -> Firestore -> admin rendeléslista folyamat.

6.1. Modulok

7. táblázat - Fő modulok és kapcsolataik

Modul	Feladat	Kapcsolódó adat
Home / Product UI	Kezdőlap, kategóriák, termékkártyák.	products

Cart	Kosar állapot, mennyiség, összegzés.	local state, products
Checkout	Adatbekeres, validacio, rendelés létrehozása.	orders
Profile	Felhasználói adatok és rendelések.	users, orders
Admin	Termék, rendelés, készlet és vásárló kezelése.	products, orders, savedCustomers
Invoice	PDF bizonylat generalasa.	orders, order items
AI assistant	Katalogushoz kötött válaszadás.	products, Worker proxy

[KODRESZLET HELYE] `src/pages/checkout/checkout.ts` - kb. 367. sortól

Mi legyen látható a kódreszletben: A rendelés véglegesítésének folyamata, validáció és adatösszeállítás.

[KODRESZLET HELYE] `src/app/services/order.service.ts` - 41-127. sor

Mi legyen látható a kódreszletben: Statusz, audit és készletváltozás tranzakciós kezelése.

[KODRESZLET HELYE] `workers/openrouter-proxy/src/index.js` - teljes proxy lényegi része

Mi legyen látható a kódreszletben: OpenRouter hívás szerveroldali kulcskezeléssel és CORS korlátozással.

7. Adatmodell

A Firestore dokumentumalapú adatmodellje nem relációs táblakból, hanem gyűjteményekből és dokumentumokból épül fel. Ez gyors fejlesztést tesz lehetővé, de megköveteli, hogy az entitások és mezők tudatosan legyenek meghatározva. A TDLWebshop esetében a legfontosabb entitások a termék, kosartétel, rendelés, felhasználói profil, mentett vásárló, kupon, számla és auditbejegyzés.

Az adatmodell tervezésénél fontos szempont volt, hogy a vásárlói és admin felület ugyanarra az adatbázisra épüljön. A termékeket a vásárló listaként látja, az admin pedig szerkesztheti, importálhatja vagy készlet szempontból vizsgálhatja. A rendeléseket a vásárló saját profiljából követi, az admin pedig státuszt és teljesítést kezel rajtuk.

A dokumentumalapú modell egyik kockázata, hogy a túl laza szerkezet adatminőségi problémákhoz vezethet. Emiatt a Firestore rules és az alkalmazás oldali validáció különösen fontos: nem elég az adatot elmenteni, azt is ellenőrizni kell, hogy a mezők megfelelő típusúak és jogosultsággal írhatók.

8. táblázat - Adatmodell fő entitásai

Entitas / gyujtemeny	Fontos mezok	Kapcsolat / szerep
products	name, sku, category, price, stock, images, discount	Termeklista, admin termékkezeles, AI katalogus.
orders	customer, items, total, status, paymentMethod, createdAt	Checkout és admin rendeléskezeles.
orderStatusAudit	orderId, oldStatus, newStatus, actor, createdAt	Statuszváltozás nyomon követése.
users	email, role, disabled, profile data	Jogosultság és profiladatok.
savedCustomers	name, email, phone, company, taxNumber, disabled	Helyszini vásárlás és mentett vásárló kezelése.
coupons	code, discount, active, validity	Kedvezmény logika.
newsletter	email, createdAt	Hírlevel feliratkozás admin oldali láthatósága.

[KEP / ABRA HELYE] Adatmodell diagram

Beillesztendő tartalom: Mermaid vagy PlantUML diagram a products, orders, users, savedCustomers, coupons és audit kapcsolataival.

[KODRESZLET HELYE] `src/app/services/order.service.ts` - 269-302. sor

Mi legyen látható a kódreszletben: Számlaszám általános logikája és rendeléshez kapcsolása.

8. Megvalósítás kulcsfolyamatai

A megvalósítás fejezet célja, hogy ne csak a használt technológiákat sorolja fel, hanem bemutassa a rendszer legfontosabb működési pontjait. A TDLWebshop esetében ilyen kulcsfolyamat a checkout, a rendelések státuszkezelése, a helyszini vásárlás, a PDF bizonylat, a CSV termékimport, a jogosultságkezelés és az AI asszisztens.

A checkout folyamatban a felhasználó adatai, a kosár tartalma, a szállítási és fizetési mód, valamint az esetleges kupon egy közös rendelési objektummal áll össze. A rendszer validálja a kötelező mezőket, majd a rendelés létrehozásával a vásárlói út lezárul. A sikeres rendelés után a vásárló visszajelzést kap, az admin pedig látja a rendelést.

A rendelés státuszkezelésnél fontos mérnöki szempont, hogy a státuszváltás ne elszigetelt mezőmódosítás legyen. A rendszer auditbejegyzést készít, és bizonyos esetekben készletváltozást is végrehajt. Ez a logika a dolgozatban külön hangsúlyozható, mert megmutatja, hogy a rendszer az üzleti következményeket is figyelembe veszi.

8.1. Checkout és rendelésmentes

A checkout a vásárlói oldal legkritikusabb folyamatainak egyike. Itt derül ki, hogy a termékbongeszes és kosár logika valós rendelése alakul-e. A felületen a felhasználó megadja a számlázási, szállítási és kapcsolattartási adatokat, a rendszer pedig ellenőrzi az email és telefonszám formátumát is.

MVP-korlatként fontos megemlíteni, hogy a webes rendelés esetében bizonyos ár- és kedvezményadatok kliensoldaltól érkeznek. Ez szakdolgozati prototípussal elfogadhatóan dokumentálható, de eless webshopban szerveroldali újraszámolásra lenne szükség. A dolgozatban ezt nem elrejténi kell, hanem mérnöki tudatossággal leírni.

[KODRESZLET HELYE] src/pages/checkout/checkout.ts - kb. 367. sortól

Mi legyen látható a kódreszletben: Rendelés véglegesítése, checkout adatok összeállítása és hibaágak.

8.2. Státusz, audit és készlet

A rendelések teljesítése és törlése készletoldali következményekkel járhat. A helyes megoldás célja, hogy a rendelés állapota, az auditbejegyzés és a készletlogika összhangban legyen. Ez szakdolgozati szempontból az egyik legerősebb rész, mert nem pusztán felületi CRUD művelettel van szó.

[KODRESZLET HELYE] src/app/services/order.service.ts - 41-127. sor

Mi legyen látható a kódreszletben: Státuszváltás, auditbejegyzés és készlet tranzakciós kezelése.

8.3. Helyszini vasarlas

A helyszini vasarlas funkcio a rendszer domainhez illeszkedo kulonlegessege. Az admin vagy dolgozo mentett vasarlot valaszthat, uj adatokat adhat meg, termeket kereshet, mennyiseget allithat, majd a vasarlas utan PDF bizonylatot tolthet le. Ez a folyamat a B2B es boltban torteno ertekesitesi helyzeteket modellezi.

[KODRESZLET HELYE] src/app/services/order.service.ts - 222-267. sor

Mi legyen lathato a kodreszletben: Helyszini rendeles tranzakcios mentese es termektetelek kezelese.

8.4. CSV import

A termekfeltoltesnel a kezi rogzitest kiegésziti a CSV import. Ez kulonosen akkor hasznos, ha tobb tucát termeket kell feltolteni kepekkel, arakkal, SKU-val es kategoriakkal. A validacios lepes azert fontos, mert importnal egy hibas oszlop vagy elvalaszto karakter sok dokumentumot ronthatna el.

[KODRESZLET HELYE] src/pages/admin/admin.ts - 1181-1257. sor

Mi legyen lathato a kodreszletben: CSV import validacio, hibas sorok jelzese es mentetho termekek feldolgozasa.

8.5. PDF bizonylat

A PDF bizonylat/szamla generalasa az admin folyamatok egyik lathato eredmenye. A dokumentum tartalmazza a kiallito es vevo adatait, a rendeles azonositojat, a tetelek listajat, a netto es brutto osszegeket, valamint a fizetesi es szallitasi adatokat.

[KODRESZLET HELYE] src/app/services/invoice.service.ts - 9-154. sor

Mi legyen lathato a kodreszletben: PDF szamla felépítése, fejlec, vevo/kiallito blokkok, tetelsorok es osszesites.

[KEP / ABRA HELYE] General PDF bizonylat

Beillesztendo tartalom: A PDF-et nyisd meg es keszits kepernyokepet ugy, hogy a fejlec, vevo es tetelsorok latszodjanak.

8.6. AI asszisztens

Az AI asszisztens a saját termekkatalogusból kiindulva próbál segítséget adni. A cél az, hogy a válasz ne legyen független a webshop adataitól. Ha nincs pontos találat, a

rendszernek inkább altalanos szakmai iranyt kell adnia, es jeleznie kell, hogy pontos ajanlathoz email es vagy személyes egyeztetes szukseges.

[KODRESZLET HELYE] src/app/services/chatbot-llm.service.ts - 26-84. es 217-236. sor

Mi legyen lathato a kodreszletben: AI domainkorlatozas, kataloguslogika es nem relevans kerdesek kezelese.

9. Biztonsag es adatvedelem

A webes rendszerekben a biztonsag nem kiegészito elem, hanem alapkovetelmeny. A TDLWebshop esetenben kulonosen fontos a felhasznaloi adatok, rendelési adatok, admin jogosultsagok, kuponok, PDF-ek es AI API kulcsok kezelese. A dolgozatban ezeket nem eleg megemliteni: be kell mutatni, milyen kockazatot jelentenek es milyen megoldas csokkenti a kockazatot.

A Firebase/Firestore hasznalata miatt a biztonsagi szabalyok kulcsszerepet kapnak. A frontend kod onmagaban nem ved elegge, mert egy rosszindulatu kliens kozvetlenul is probalkozhat adatbazis-muveletekkel. Emiatt a Firestore rules feladata, hogy szerepkor, aktiv felhasznalo es payload szerkezet alapjan korlatozza az olvasast es irast.

A titokkezeles szempontjabol kulon fontos, hogy API kulcs vagy jelszo ne keruljon a repositoryba. Az OpenRouter kulcs a Cloudflare Worker secretjekent tarolando, a repoban pedig csak .env.example es dokumentalt konfiguracios valtozok szerepelhetnek.

9. tablazat - Biztonsagi minimum es kockazatok

Kockazat	Leiras	Kezeles a projektben	Tovabbi javaslat
Jogosulatlan admin muvelet	Normal user admin adatot modositana.	Szerepkor alapu rules es frontend guard.	Custom claim es token revocation es rendszerben.
Tiltott felhasznalo	Disabled user tovaab probalkozik.	Aktiv user ellenorzes szabalyokban.	Auth szintu tiltas eseben.
Kupon visszaeles	Kliensoldali kuponmanipulacio.	Kuponvalidacio es dokumentalt MVP-korlat.	Szerveroldali ujraszamolas.

PDF adatvedelem	Szemelyes adatok szerepelnek a bizonylaton.	Csak jogosult felületen generalható.	Letöltési naplozas es tarolasi szabaly.
AI API kulcs kiszivargas	Kulcs frontendbe vagy gitbe kerul.	Cloudflare Worker secret, .env.example.	Rate limit es kvota.
Vendeg rendelés azonositas	Email-alapu osszekotes kockazata.	Dokumentalt korlat.	Regisztralt fiokhoz kotott rendelestortenet erosítése.

[KODRESZLET HELYE] firestore.rules - 25-76. sor

Mi legyen lathato a kodreszletben: Aktiv felhasznalo, admin es dolgozo jogosultsag ellenorzes.

[KODRESZLET HELYE] firestore.rules - 294-361. sor

Mi legyen lathato a kodreszletben: products, orders, users, savedCustomers es audit szabalyok.

[KODRESZLET HELYE] src/pages/admin/admin.ts - 607-748. sor

Mi legyen lathato a kodreszletben: Admin es dolgozoi jogosultsagok feluleti kezelese.

9.1. MVP-korlatok biztonsagi szempontbol

A szakdolgozati rendszer nem vállalja egy teljes erteku penzugyi backend szerepet. A bankkartya elfogadas, szerveroldali arujraszamolas, keszletfoglalas es ERP-szintu integracio tovaabfejlesztési irány. Fontos, hogy ez nem hiba, ha a dolgozatban vilagosan szerepel mint tudatos MVP-hatar.

Az AI asszisztensnel a CORS beallitas onmagaban nem teljes visszaeles elleni vedelem. Ezt rate limit, kvota vagy felhasznalo azonositashoz kotott hivaslimit erositheti. A dolgozatban ezt a reszt ugy erdemes bemutatni, mint mukodo prototipust, amelynel elesites elott tovaabbi vedelmi lepesek kellenek.

10. Teszteles es validacio

A teszteles celja annak bizonyitasa, hogy a fo felhasznalo es adminisztracios folyamatok nem csak elmeletben leteznek, hanem ellenorizhetően mukodnek. A TDLWebshop eseteben a teszteles ket szinten tortent: automata tesztekkel es kezi tesztjegyzokonyvvel.

Az automata tesztek elonye, hogy a CI folyamatban is futtathatok, így egy GitHub Actions zöld állapot kulso bizonyitek a build es tesztek sikersegere. A kezi teszteles ezzel szemben a teljes felhasznaloi elmenyt, a kepernyoallapotokat, hibauzeneteket es reszponzivitast vizsgalja.

A dolgozatban kulonosen fontos megmutatni, hogy a kritikus folyamatok ellenorizve voltak: checkout, rendelés létrejött, készletváltozás, admin státuszváltás, tiltott jogosultság, kuponvalidáció, CSV import és AI asszisztens.

10. táblázat - Tesztesi terv összefoglalása

Tesztazonosito	Folyamat	Elvart eredmény	Bizonyitek
T1	Regisztráció és beépés	Felhasználó profilja létrejön, tiltott user üzenetet kap.	Kezi teszt
T2	Termekkeresés	A lista szűrodik, nincs találat esetén üres állapot látszik.	Kezi teszt
T3	Kosár mennyiség	Összeg frissül, törles működik.	Automata/kezi
T4	Checkout validáció	Hibas email/telefon nem enged tovább.	Kepernyokep
T5	Sikerés rendelés	Rendelés létrejön Firestore-ban és adminban látszik.	Kezi teszt
T6	Admin státuszváltás	Audit és készletváltozás rögzödik.	Automata/kezi
T7	CSV import	Hibas sorok jelzodnek, ervenyos sorok menthetoeok.	Kezi teszt
T8	PDF bizonylat	PDF tartalmazza az adatokat és nem csuszik össze.	Kepernyokep
T9	AI asszisztens	Relevans kérdésre domainválasz, irrelevansra elhatárolás.	Kezi teszt

T10	CI	GitHub Actions build/test zold.	Kepernyokep
-----	----	------------------------------------	-------------

10.1. Build es CI

A projekt CI folyamata GitHub Actions segítségével fut. A workflow célja, hogy friss commit után ellenőrizze a telepíthetőséget, buildet és teszteket. A zöld CI állapot a dolgozatban külön képernyőképként szerepeljen, mert gyorsan bizonyítja, hogy a repo nem csak lokális gépen működik.

[KEP / ABRA HELYE] GitHub Actions zöld CI futás

Beillesztendő tartalom: GitHub Actions oldalon a legfrissebb zöld workflow run látszódjon, commit névvel és időtartammal.

10.2. Kezi tesztjegyzék

A kezi tesztjegyzék a dolgozat mellékletében vagy a tesztelés fejezetben érdemes bemutatni. A tesztelés során nem csak a sikeres utakat, hanem a hibaágakat is ellenőrizni kell. Ilyen példa a hibás email, tiltott felhasználó, készlethiány, jogosulatlan admin művelet vagy nem releváns AI kérdés.

[KEP / ABRA HELYE] Checkout validációs hiba

Beillesztendő tartalom: Checkout oldal hibás email vagy telefonszám beírásával.

A vásárlói út részletes értelmezése

A vásárlói út a kezdőlapra indul, ahol a felhasználó gyorsan képet kap a webshop témájáról és a főbb kategóriákról. A keresősáv kiemelt szerepet kap, mert egy épületgépészeti webshopban a vásárló sokszor konkrét terméknevet, méretet vagy cikkszámot keres. A kategóriák dropdown formája egyszerre ad gyors navigációt és modern felületi eleményt.

A terméklista célja az, hogy a vásárló össze tudja hasonlítani a kínálatot. A termékkartyákon az ár, készlet és terméknev együtt jelenik meg. A kedvezményes vagy új termékek jelölése segíti a döntést, de a felület nem viszi túlzásba a vizuális elemeket, mert a muszáj termékeknel az olvashatóság fontosabb, mint a pusztán reklám jellegű megjelenés.

A kosar oldal a döntés és a rendelés közötti átmenet. Itt a felhasználó ellenőrizheti a mennyiséget, törölhet tételt, majd továbbléphet a checkoutra. A kosar logikája azért fontos, mert az összegzésnek mindig követnie kell a mennyiségváltozást, és nem szabad instabil termékazonosítókra épülnie.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

Adminisztrációs folyamatok részletes értelmezése

Az admin felület nem egyszerű másolata a vásárlói oldalnak. Más célcsoportot szolgál ki, ezért surubb információs elrendezést, gyors műveleti gombokat és több adatot tartalmaz. Egy adminnak nem marketinges termékbemutatót kell látnia, hanem olyan adatokat, amelyek alapján rendelést, készletet és vásárlót tud kezelni.

A helyszíni vásárlás funkció különösen fontos a projektben, mert egy valószínűleg épületegyesületi boltban a rendelések nem mindig weben érkeznek. Az adminnak vagy dolgozonak gyorsan ki kell tudnia választani egy mentett vásárlót, terméket kell keresnie, mennyiséget kell megadnia, majd bizonylatot kell generálnia. Ez a funkció emiatt a domainhez illeszkedő, nem általános webshopos kiegészítés.

A dolgozói jogosultságok bevezetése azt mutatja, hogy a rendszer nem egyetlen admin felhasználóra épül. A dolgozó tud terméket és készletet kezelni, vásárlást rögzíthet, de nem kap minden jogosultságot. Ez közelebb viszi a rendszert egy valószínű szervezeti működéshez.

A végleges dolgozatban ezt a részt érdemes saját tapasztalatokkal kiegészíteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a védesen hogyan tudnád saját szavaiddal elmagyarázni.

Adatbiztonsági és minőségi szempontok részletes értelmezése

A Firestore szabályokkal kapcsolatban fontos felismerés, hogy a kliensoldali ellenőrzés csak felhasználói kényelmi és UX szempontból elég. Biztonsági szempontból az adatbázis oldali szabályok jelentik az igazi védelmi vonalat. Emiatt a szerepkörök, aktív felhasználói állapot és payload mezők ellenőrzése szakdolgozati szinten is értékes része a projektnek.

A titkok kezelése külön fejezetet érdemel, mert modern webes fejlesztéskor az API kulcsok és jelszavak véletlen commitolása gyakori hiba. A projektben a végleges irány az, hogy a

repoban csak pelda konfiguracio szerepel, az eles kulcsok pedig szolgáltatoi secretben tarolodnak. Ez mernoki es biztonsagi szempontbol is helyes irany.

A teszteles es CI nem csak technikai formalitas. A zold CI azt bizonyitja, hogy a projekt egy kulso futtatokornyezetben is le tud fordulni es a tesztek lefutnak. Ez kulonosen fontos a konzulensi visszajelzes alapjan, mert a repo rendezettsége es reprodukhalhosaga a vegleges ertekelesben is szerepet kap.

A vegleges dolgozatban ezt a reszt erdemes saját tapasztalatokkal kiegesziteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a vedesen hogyan tudnád saját szavaiddal elmagyarázni.

Tovabbfejlesztési lehetosegek reszletes ertelmezese

Eles kornyezetben a legfontosabb továbbfejlesztés a szerveroldali üzleti logika erősítése lenne. A webes rendelesek arat, kedvezményét és készletet nem ideális teljes mértékben kliensoldali adatokra bízni. Egy Cloud Function vagy külön backend endpoint újraszámolhatná a végösszeget és ellenőrizhetné a készletet.

Az AI asszisztens továbbfejlesztéséhez hasznos lenne keresési index, termékleírásokból készült strukturált tudástár és rate limit. Ez csökkentene a pontatlan ajánlások és visszaelekések kockázatát. A rendszer jelenlegi állapotában prototípusnak tekinthető, amely megmutatja az irányt, de nem helyettesíti a szakmai ügyfélszolgálatot.

A készletfigyelés tovább bővíthető fogási trenddel, utánrendelési javaslattal és admin értesítésekkel. Ez különösen jól illeszkedik az épülelgepeszeti kereskedelemhez, ahol bizonyos szereleségi anyagok és gyakran használt alkatrészek készleten tartása üzletileg fontos.

A vegleges dolgozatban ezt a reszt erdemes saját tapasztalatokkal kiegesziteni: milyen hibát találtál, mit javítottál, milyen döntést hoztál, és a vedesen hogyan tudnád saját szavaiddal elmagyarázni.

Reszletes szakdolgozati kifejtések az osszedolgozott munkapeldanyhoz

Az alábbi alfejezetek azt a célt szolgálják, hogy a dolgozat ne csak rövid fejezetváz legyen, hanem a konzulensi elvárásoknak megfelelően részletesen is bemutassa a rendszer szakmai háttérét, döntéseit, korlátait és bizonyítékait. A végleges változatban ezeket a részeket saját megfogalmazásra kell húzni, de tartalmilag a TDL Webshop mernoki történetét követik.

A projekt valós felhasználói problémája

A TDLWebshop alap gondolata nem pusztán az volt, hogy készüljön egy termékeket listázó weboldal. Az épütelgépészeti kereskedelemben a vásárlói és üzemeltetői folyamatok egyszerre jelennek meg: a vásárló terméket keres, rendelést ad le, kedvezményt használ, a dolgozó pedig készletet ellenőriz, rendelést rögzít és bizonylatot készít. Ez a kettős igény adta a projekt szakdolgozati értékét, mert a rendszernek nem egyetlen oldalt, hanem több, egymással összefüggő munkafolyamatot kellett kezelnie.

A dolgozatban ezt azért fontos részletesen bemutatni, mert egy webshop csak akkor tekinthető termékszerű MVP-nek, ha a felhasználó nem akad el a külső folyamatokban. A kezdőlap, terméklista, kosár és checkout onmagában még nem elég, ha nincs mellette adminisztráció, rendeléstörténet, jogosultságkezelés és készletlogika. A TDLWebshop ezek közül több területet is összekapcsol, ezért a projekt közelebb áll egy valósi üzleti alkalmazáshoz, mint egy egyszerű tanuloprojekthez.

A valósi probléma másik oldala az, hogy az épütelgépészeti termékeknel a vásárló sokszor nem csak nevet vagy árat néz. Fontos lehet a kategória, a műszaki jellemző, a készlet, a szállítási lehetőség és az, hogy helyszíni vásárlás során is rögzíthetők legyenek a tételtek. Emiatt a rendszer nem csak látványos felületet kapott, hanem olyan admin oldali funkciókat is, amelyekkel a bolt munkafolyamatai bizonyíthatók.

A végleges dolgozatban ide érdemes beilleszteni egy rövid saját történetet arról, miért pont épütelgépészeti webshop lett a téma. Ez segít abban, hogy a dolgozat ne általános szoftverleírásnak hasson, hanem látszódjon mögötte a személyes motiváció és a domain ismerete.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne onmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Az MVP határának indoklása

A szakdolgozati MVP határa tudatosan úgy lett kijelölve, hogy a rendszer a vásárlói utat és az adminisztrációs utat egyaránt bizonyítsa. Nem célja a projektnek, hogy teljes értékű vállalatirányítási rendszer, bankkártya-elfogadó rendszer vagy számlázó program legyen. A cél az, hogy a rendeléstől a készlet- és admin kezelésig bemutatható legyen egy működő, bővíthető tervezett webshop alap.

Ez a határ azért vedhető, mert a szakdolgozat nem egy eles kereskedelmi szolgáltatás teljes jogi és pénzügyi megfelelést vállalja, hanem egy mernokileg indokolt MVP-t. A bankkártya fizetés például nem teljes online fizetési integráció, hanem a fizetési módok modellezése és a rendelési folyamat része. A PDF bizonylat sem hivatalos e-számla szolgáltatás, hanem a rendelest összefoglaló dokumentum, amely bemutatja a generált kimenet logikáját.

Az MVP-határ része az is, hogy bizonyos kockázatokat a dolgozat korlátként nevez meg. Ide tartozik a webes rendelesek szerveroldali végösszeg-újraszámítása, az AI asszisztens pontos termékajánlasi korlátja, valamint az eles fizetési szolgáltató hiánya. Ezek nem elhallgató hibák, hanem olyan továbbfejlesztési pontok, amelyek egy szakdolgozatban kifejezetten jól mutatják a realis onterkelest.

A bírálo számára különösen hasznos, ha az MVP-határ táblázatban is megjelenik: mi készült el, mi maradt prototípus, és mi tudatosan nem lett bevállalva. Ezzel elkerülhető, hogy a projekt felkész rendszernek hasson, miközben valóban meghatározott célra készült.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne onmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált dönteshez kapcsolódjon.

Piaci összehasonlítás szerepe

A piaci összehasonlítás célja nem az, hogy a TDLWebshop minden nagy webáruháznál jobb legyen. A cél az, hogy látszódjon: milyen megoldások jellemzőek a hasonló webshopokra, és ehhez képest a saját rendszer milyen szakdolgozati többletet ad. A konzulensi visszajelzés alapján ez a fejezet különösen fontos, mert megmutatja, hogy a projekt nem elszigetelt kódgyakorlat.

Az összehasonlításban érdemes 2-4 épülelgepeszeti vagy muszaki webshopot vizsgálni. A szempontok lehetnek: termékkeresés, kategória struktúra, kosár, checkout, regisztrált profil, admin jellegű funkciók láthatósága, készletinformáció, szakmai támogatás, akciós termékek és mobilos használhatóság. A saját rendszer értéke nem feltétlenül abban van, hogy több terméket tartalmaz, hanem abban, hogy a vásárlói és admin oldali funkciókat egy szakdolgozati MVP-ben összekapcsolja.

A TDLWebshop egyik erősebb pontja a helyszíni vásárlás rögzítése, a mentett vásárlók kezelése és a dolgozói szerepkör megjelenése. Ezek olyan funkciók, amelyek sok publikus

webshopon nem lathatok, mert belso vállalati feluletekhez tartoznak. Szakdolgozati szempontbol viszont pont ezek bizonyitjak, hogy a rendszer nem csak kirakatoldal.

A piaci fejezetet a vegleges dolgozatban egy tablazattal erdemes zarnod. A tablazatban ne csak pipakat használj, hanem rovid megjegyzeseket is: miért fontos az adott szempont, és a TDLWebshop hogyan kezeli.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepnyokep, rovid kodreszlet vagy tablazat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

Kovetelmenyek es traceability

A kovetelmenyek fejezete kot össze a celkituzes es a megvalositas kozott. Ha egy rendszerben sok funkcio van, konnyen elveszik, hogy melyik mire szolgalt. A traceability tablazat ezt oldja meg: kovetelmeny, use case, megvalositott modul es teszt bizonyitek egy sorban latszik.

A TDLWebshop eseteben kulon kell kezelni a vasarloi kovetelmenyeket, az adminisztracios kovetelmenyeket es a biztonsagi kovetelmenyeket. A vasarloi oldalhoz tartozik a termekkereses, kosar, checkout, profil es rendelestortenet. Az admin oldalhoz tartozik a termekfeltoltes, CSV import, rendelkezes, keszletfigyeles, helyszini vasarlas, mentett vasarlok es jogosultsagkezeles.

A biztonsagi kovetelmenyek nem csak technikai reszletek. Ide tartozik, hogy tiltott felhasznalo ne tudjon tovabb rendelni, normal vasarlo ne erjen el admin funkciokat, a dolgozo csak a saját szerepkorenek megfelelo feluletet lassa, es API kulcs ne szerepeljen a repoban. Ezeket a dolgozatban kulon erdemes kifejteni, mert a webes rendszereknel gyakori biralati pont a jogosultsagkezeles.

A traceability tablazat azert is jo, mert a teszteles fejezethez is hidat ad. Ha leirod, hogy peldaul a checkout validaciohoz melyik kepnyokep es melyik teszt tartozik, akkor a biralo gyorsan latja, hogy nem csak allitod a mukodest, hanem bizonyitod is.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepnyokep, rovid kodreszlet vagy tablazat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

GUI/UX döntések indoklása

A felület tervezésénél a cél egy modern, ipari hangulatu webshop volt. A dark mode, a kek-piros accent színek és a TDLWebshop logo világa együtt adja azt a vizuális irányt, amely az épületegészeti témához illik. A fontos döntés az volt, hogy a látvány ne menjen az olvashatóság rovására.

A kezdőlap szerepe az első benyomás. Itt a kategória dropdown, a keresősáv, a hero szekció és a kiemelt termékek vezetik a felhasználót. A terméklista és termékkártya részeknél már a funkcionalitás erősebb: terméknev, ár, készlet, kosárba helyezés és részletek gomb. A dolgozatban a képernyőképeken keresztül érdemes megmutatni, hogy ugyanaz a vizuális rendszer több oldalon is visszatér.

A light mode külön kihívás volt, mert nem elég a háttérre fehérre és a szöveget feketére váltani. A kontraszt, gombok, fejléc és kártyák arnyékolása külön figyelmet igényel. Ezt a dolgozatban UX döntésként lehet leírni: a layout azonos marad, de a színek és arnyékok alkalmazkodnak a témához.

Az admin felületnél a design más szempontot követ. Itt a sürűbb információ, a gyors gombok, táblázatok, filterek és státuszok a fontosak. A dolgozatban jó, ha külön képernyőkép van a vásárlói oldalról és az admin oldalról, mert így látszik, hogy két célcsoporthoz más felületi logika tartozik.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kód részlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Angular komponensalapú felepítés

Az Angular használata azért indokolt, mert a rendszer több oldalból, szolgáltatásból és állapotból áll. A komponensek és service-ek elkülönítése segít abban, hogy a megjelenítés és az adatkezelés ne keveredjen teljesen össze. Egy webshopnál ez különösen fontos, mert ugyanazt a termékadatot több helyen is használni kell.

A terméklista, kosár, checkout, profil és admin oldalak külön felelőségeket kapnak. A service retegek, például a product, cart, order, auth vagy chatbot service, az adatbetöltés és üzleti logika közelebbi részeit kezelik. Ez a felepítés a dolgozat architektúra fejezetében jól bemutatható, mert látszik belőle a kliensoldali alkalmazás szerkezete.

A komponensalapú felbontás másik előnye a tesztelhetőség. Bizonyos részek önállóan vizsgálhatók, például a kosárösszegezés, kuponlogika vagy validáció. A szakdolgozatban nem kell minden tesztet részletesen bemutatni, de a kritikus folyamatokhoz érdemes egy-egy példát hozni.

A végleges szövegben érdemes röviden kitérni arra is, hogy az Angular választása milyen alternatívákhoz képest történt. Például React vagy Vue is alkalmas lett volna, de az Angular struktúráltsága és service-orientált felépítése illeszkedett a nagyobb alkalmazás jellegű projekthez.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódrészlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Firestore és Firebase szerepe

A Firebase a projektben backend-szerű szolgáltatásokat ad: hitelesítés, Firestore adatbázis, hosting és biztonsági szabályok. Ez lehetővé tette, hogy a szakdolgozat fókuszba ne teljesen egy saját backend infrastruktúrára menjen el, hanem a webshop üzleti folyamataira és adatmodelljére.

A Firestore dokumentumalapú modellje jól illeszkedik a termékek, rendelések, felhasználói profilok, kuponok és mentett vásárlók tárolásához. Ugyanakkor a dokumentumalapú adatbázis nem ugyanazt a gondolkodást igényli, mint egy relációs adatbázis. A dolgozatban érdemes megmutatni, mely entitások külön collectionbe kerültek, és milyen adatok kapcsolják őket össze.

A biztonsági szabályok külön fejezetet érdemelnek. A kliensoldali kód önmagában nem véd elegendő, mert egy rosszindulatú felhasználó megpróbálhat közvetlenül adatbázishoz fordulni. Emiatt a Firestore rules oldalon is kezelni kell, ki mit olvashat és írhat. Ez a projekt egyik erősebb mérnöki pontja.

A Firebase config és API kulcs kérdése külön magyarázatot igényel. Webes Firebase alkalmazásoknál a kliensoldali config részben publikus, de ettől még a valódi védelem a security rules és a jogosultsági modell. A dolgozatban érdemes leírni, hogy titkok, OpenRouter kulcsok és jelszavak nem kerülhetnek repóba.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepernyokep, rovid kodreszlet vagy tablázat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

Rendeleskezeles es keszletlogika

A rendeleskezeles a webshop egyik legfontosabb folyamata. A vasarloi oldalon a checkout gyujti ossze a kosarat, szallitasi adatokat, fizetesi modot es kedvezmenyt. Az admin oldalon a rendeles statusza kovetheto, modositatho, es a keszletvaltozas is kapcsolodik hozza.

A helyszini vasarlasnal kulonosen fontos volt, hogy a rendeles, keszletvaltozas es bizonylat ne teljesen kulon mozogjon. Ha egy admin rogzit egy eladast, a rendszernek figyelembe kell vennie a kivalasztott termekeket, mennyisegeket es vasarlot. Ez a folyamat szakdolgozati szempontbol ertekesebb, mint egy egyszeru kosarmentes, mert belso bolti munkafolyamatot modellez.

A webes rendelesehez kapcsolodo korlatot oszinten erdemes leirni. Ha bizonyos arak es kedvezmenyek kliensoldalrol jonnek, akkor eles rendszerben szerveroldali ellenorzesre lenne szukseg. Ez nem feltetlenul gyengiti a szakdolgozatot, ha a korlat vilagosan szerepel es tovaabfejlesztési terv is tartozik hozza.

A statuszvaltas es audit naplo azt mutatja, hogy a rendeles állapotvaltozasai kovethetok. Ez a dolgozatban jo pelda arra, hogyan lehet egy uzleti folyamatot nem csak vegeteredmenykent, hanem tortenettel együtt kezelni.

Kapcsolodo bizonyitek a vegleges dolgozatban: ide kerulhet kepernyokep, rovid kodreszlet vagy tablázat, amely az adott allitast alatasztja. A cel, hogy a szoveg ne onmagaban alljon, hanem lathato rendszerreszhez, teszthez vagy dokumentalt donteshez kapcsolodjon.

PDF bizonylat es szamlageneralas

A PDF generalas a projektben a rendelési folyamat kezzelfoghato kimenete. A vasarlo vagy admin nem csak adatbazis-bejegyzest kap, hanem letoltheto dokumentumot is. Ez szakdolgozati szempontbol jól demonstralja, hogy a rendszer képes strukturalt adatból dokumentumot eloallitani.

A bizonylat felepítése tartalmazza a rendeles azonositojat, vevői adatokat, termekeket, fizetési es szallitasi modot, valamint a vegosszeget. A korabbi visszajelzes alapján fontos

volt az elrendezés javítása, hogy az összeg ne csusszon bele más blokkokba. Ezt a dolgozatban akár fejlesztési tapasztalatként is meg lehet említeni.

Eles környezetben egy jogilag érvényes e-számla más követelményeket jelentene. Ehhez külső számlázó szolgáltatás, jogszabályi megfelelés és adatszolgáltatási logika kellene. A TDLWebshop jelen állapotában PDF bizonylatot generál, ami MVP szinten bemutatja a dokumentumkészítés elvét.

A végleges dolgozatban ide mindenképpen kell egy kép a generált PDF-ről. A kép mellett röviden magyarul el, mely adatok a rendelesebből származnak, és melyek fix cégeadatok vagy számlaformázási elemek.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált dönteshez kapcsolódjon.

AI asszisztens korlátai és szerepe

Az AI asszisztens nem azért került a rendszerbe, hogy szakembert vagy ügyfélszolgálatot teljesen helyettesítsen. A cél az volt, hogy a termékkatalógushoz és épületegészeti témához kapcsolódóan segítse a tájékozódást. Ez fontos különbség, mert egy szakdolgozati prototípusban az AI-használatot korlátokkal együtt kell bemutatni.

A rendszerben az OpenRouter hívás nem közvetlenül a kliensből történik, hanem Cloudflare Worker proxyon keresztül. Ez azért fontos, mert az API kulcs nem kerülhet a böngészőbe vagy a repóba. A dolgozatban ez biztonsági és architekturalis döntesként is szerepelhet.

A válaszoknál külön szabály, hogy az asszisztens ne ajánljon random terméket akkor, ha nincs megfelelő katalógus-találat. Ilyenkor adhat általános szakmai irányt, de jeleznie kell, hogy pontos ajánlatot vagy beszerezhetősegért emailben vagy személyesen érdemes egyeztetni. Ez csökkenti a téves termékajánlás kockázatát.

A dolgozatban külön kell választani a fejlesztés során használt AI-t és a webshopba beépített AI asszisztensét. Az előbbi munkaeszköz volt, az utóbbi a kész rendszer egyik funkciója. Ez a különválasztás megfelel a konzulensi elvárásnak is.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszt vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált dönteshez kapcsolódjon.

Tesztelesi bizonyítás és kezi ellenőrzés

A teszteles fejezet célja, hogy a rendszer működését ne csak állítások támasszák alá. A build, az automata tesztek és a kezi tesztjegyzék egyaránt adnak bizonyítékot. Ez különösen fontos, mert a szakdolgozatban több felhasználói szerepkör és több kritikus folyamat szerepel.

Az automata tesztek közül érdemes kiemelni a kosár, checkout/kuponlogika, admin státuszváltás, PDF generálás, AI asszisztens és validáció tesztjeit. Nem kell minden assertet bemutatni, de a dolgozatban legyen egy táblázat: tesztelt funkció, módszertan, várt eredmény, tényleges eredmény.

A kezi tesztek a vizuális és folyamatbeli részek miatt szükségesek. Ilyen példa a mobilos megjelenés, admin táblázatok kezelhetősége, checkout hibajelzések, mentett vásárló kiválasztása vagy PDF letöltés. Ezeket screenshotokkal és pipálható tesztlistával lehet igazolni.

A GitHub Actions zöld CI képernyőképe erős bizonyíték, mert megmutatja, hogy a projekt nem csak a saját gépen futott. A végleges dolgozatban ezt a reprodukálhatósági fejezethez érdemes tenni.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Repo-higiénia és beadáshoz tartozó tisztaság

A konzulensi visszajelzés egyik fő pontja a repo rendezettsége volt. Ez nem mellékes adminisztráció, hanem a szakdolgozat bírálatának feltétele. Ha a repóban `node_modules`, build mappák, jelszavak vagy helyi segédfájlok vannak, akkor a projekt kevesbé professzionálisnak tűnik.

A végleges repóban a futtatható kód, konfigurációs minták, dokumentációnak és teszteknek kell szerepelniük. A személyes segédanyagok, hosszú átirati alapok és munkapeldányok lokálisan maradhatnak, de nem kell őket GitHubra feltölteni. Ez különösen fontos, mert a dolgozat saját nyelvezetre huzása a hallgató feladata.

A `.env.example` szerepe az, hogy megmutassa, milyen környezeti változókra van szükség, de ne tartalmazzon valódi kulcsot. Az OpenRouter API kulcsot a Worker secret kezeli, nem a repóban tárolt fájl. Ez a titokkezelésről szóló részen jó példa.

A README-nek olyan személynek is segítenie kell, aki nem vett részt a fejlesztésben. Telepítés, indítás, build, teszt, Firebase, Worker és demo szerepkörök: ezek legyenek egyértelműek. A dolgozat reprodukálhatósági fejezete erre hivatkozhat.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Sajat értékelés és tanulságok

A dolgozat végi reflexió nem egyszerű lezáró bekezdés. Itt kell megmutatni, hogy mit tanultál a projektből, milyen döntések voltak nehezek, és hol látod a rendszer korlátait. A jó szakdolgozat nem hibamentesnek mutatja a projektet, hanem realisan értékeli.

Érdekes tanulság lehet, hogy a webshop fejlesztése közben a látványos felület csak az egyik rész. Legalább ilyen fontos lett a jogosultságkezelés, Firestore szabályok, tesztek, adatmodell és dokumentáció. Ez jól mutatja, hogy egy szoftveres projektben a kód mellett a bizonyíthatóság is mernoki munka.

A nehézségek közé bekerülhet a checkout validáció, a PDF elrendezés, a helyszíni vásárlás mentése, a CSV import, az AI proxy biztonságos bekötése vagy a GitHub CI stabilizálása. Ezek konkrét, védesen is jól elmondható fejlesztési tapasztalatok.

A továbbfejlesztési irányok között reális marad a szerveroldali rendelellenőrzés, éves fizetési szolgáltató, fejlettebb készlet-előrejelzés, admin riportok és AI asszisztens rate limit. Ezeket nem kell most mind megvalósítani, de jól zárják a dolgozatot.

Kapcsolódó bizonyíték a végleges dolgozatban: ide kerülhet képernyőkép, rövid kódreszlet vagy táblázat, amely az adott állítást alátámasztja. A cél, hogy a szöveg ne önmagában álljon, hanem látható rendszerreszhez, teszthez vagy dokumentált döntéshez kapcsolódjon.

Fejezetenkénti részletes kidolgozási alap

Ez a rész nem külön mellekletnek készült, hanem olyan hosszabb, szakdolgozati stílusú szövegalap, amelyet a végleges dokumentumban a megfelelő fejezetekbe lehet athelyezni vagy ott megtartani. A cél az, hogy a dolgozat terjedelme és részletezettsége közelebb kerüljön egy 40-50 oldalas szoftveres szakdolgozathoz, miközben a tartalom a TDLWebshop valós kódjához, tesztjeihez és konzulensi elvárásaihoz kapcsolódik.

Bevezetes es problemafelvetes bovites

A bevezetesben azt kell bizonyitani, hogy a TDLWebshop nem oncelu webes felulet, hanem egy valos kereskedelmi es adminisztracios problema modellezese.

A felhasznaloi oldal akkor tekintheto teljesnek, ha a vasarlo a kezdolaprol eljut a termekekig, a kosarig, majd a sikeres rendelesig.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

Az adminisztrációs oldal azért fontos, mert egy webshop életképessége nem csak a vásárlók, hanem a belső üzemeltetési folyamatok is meghatározzák.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

Az epulelgepeszeti termekkor sajátossaga, hogy a vasarlo sokszor szakmai dontesi helyzetben van, ezert a kategoria, keszlet es ajanlasi logika is lenyeges.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakedolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés kozben milyen donteshelyzet alakult ki, milyen alternativat merlegeltel, es miért a jelenlegi megoldas mellett dontottel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A projekt során a legfontosabb kompromisszum az volt, hogy a rendszer szakedolgozati MVP legyen, ne pedig minden reszleteben eles vallalati termék.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakedolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés kozben milyen donteshelyzet alakult ki, milyen alternativat merlegeltel, es miért a jelenlegi megoldas mellett dontottel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A dolgozatban a problemafelvetesnek ossze kell kotnie a szakmai domaint, a felhasznaloi igényeket es a megvalositott szoftveres megoldast.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakedolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés kozben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

MVP es kovetelmenyek bovités

Az MVP meghatározása a dolgozat egyik vedelmi pontja: világossá teszi, hogy mi készült el tudatosan, és mi maradt továbbfejlesztési irány.

A vásárlói MVP része a böngészés, keresés, kosár, checkout, rendeléstörténet és profiladatok kezelése.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakedolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőképek, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

Az admin MVP része a termékkezelés, CSV import, rendeléskezelés, készletfigyelés, mentett vásárló és helyszíni vásárlás.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőképek, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A dolgozói szerepkör megjelenése azért értékes, mert nem csak admin és vásárló közötti egyszerű jogosultsági modellt mutat be.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíts. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőképek, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A követelményeket célszerű azonosítható kódokkal ellátni, mert így a megvalósítás és tesztelés fejezeteiben vissza lehet rájuk hivatkozni.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíts. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőképek, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A követelmények között nem csak funkcionális, hanem biztonsági, használhatósági és reprodukálhatósági elvárásokat is szerepeltetni kell.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

Piaci összehasonlítás bővítés

A piaci fejezet azt mutatja meg, hogy a TDLWebshop milyen hasonló rendszerekhez képest értelmezhető, és milyen saját értéket ad.

A hasonló webshopok elemzésénél nem az a cél, hogy pontos üzleti titkokat vagy belső admin felületeket hasonlítsunk össze, hanem a publikus felhasználói élményt.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell huzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

A TDLWebshop sajátossága, hogy a publikus webshop mellett admin és dolgozói folyamatokat is bemutat, ami egy szakdolgozatban erős technikai bizonyíték.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A piaci táblázatban érdemes külön jelölni, hogy mely funkciók láthatók a vásárlói oldalon, és melyek belső üzemeltetési oldalhoz kapcsolódnak.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A mobilos használhatóság és a keresési élmény olyan szempont, amelyet a képernyőképek is jól alátámasztanak.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

A piaci elemzés végére érdemes rövid következtetést írni arról, hogy a saját rendszer nem termékmennyiségben, hanem folyamat-összefogásban erős.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDL Webshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kód-részlet is.

GUI és UX bővítés

A GUI/UX fejezetben a felületet nem csak képként, hanem döntések sorozataként kell bemutatni.

A dark mode az alap vizuális irány, mert a logo és az ipari hangulat ehhez illeszkedik legjobban.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A light mode nem külön layout, hanem ugyanannak a felületnek világosabb, üzletiesebb változata.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A termékkartyáknál az ár, készlet és kosárba helyezés gyors felismerhetősége fontosabb, mint a túlzott dekoráció.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

Az admin felületen a gyors műveletek, táblázatok és állapotjelzések dominálnak, mert ott a hatékony munkavégzés a fő cél.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A képernyőképeknek különböző állapotokat is mutatniuk kell: üres lista, hiba, sikeres rendelés, validációs hiba és jogosultsági eltérés.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

Architektúra és adatmodell bővítés

Az architektúra fejezetben azt kell megmutatni, hogyan működnek együtt az Angular komponensek, a service réteg, a Firebase és az OpenRouter proxy.

A kliensoldali alkalmazás felelős a megjelenítésért, a felhasználói interakciókért és az egyes service-ek meghívásáért.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A Firestore az adatok tárolását és a jogosultsági szabályok egy részét biztosítja, ezért az adatmodell és security rules együtt értelmezendő.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

A Cloudflare Worker az AI asszisztensnél biztonsági köztes réteg, mert az OpenRouter API kulcs nem kerülhet a kliensoldalra.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

Az adatmodellben külön figyelmet érdemel az Order és OrderItem kapcsolat, mert a kosár tartalma itt alakul tartós rendelési adattá.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

A savedCustomers és users elkülönítése azért érdekes, mert a helyszíni vásárlás és a regisztrált webshopos vásárló nem teljesen ugyanaz a fogalom.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

Megvalositas bovites

A megvalositas fejezetnek nem eleg kodreszleteket felsorolnia; azt kell bemutatnia, hogy a kod milyen uzleti folyamatot valosit meg.

A checkout folyamatban a validacio, vegosszeg-szamitas, kuponkezeles es rendelesmentes egymashoz kapcsolodo lepesek.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A statuszvaltas es audit log azt bizonyitja, hogy a rendeles nem csak statikus rekord, hanem kovetheto allapotvaltozasokkal rendelkezo entitas.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A helyszini vasarlas rogzitese az admin felulet egyik legerosebb funkcioja, mert belso bolti munkafolyamatot modellez.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A CSV import a tomeges termekfeltoltest segiti, es jól mutatja, hogy a rendszer nagyobb termekmennyisegre is felkeszített.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A PDF generalas es AI asszisztens olyan kiegészito funkciok, amelyek a projektet termekszerubb irányba viszik, de korlataikat is le kell írni.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

Biztonság és adatvédelem bővítés

A biztonsági fejezetben a hangsúly azon van, hogy a rendszer milyen visszaéléseket próbál megelőzni, és hol maradnak MVP-szintű korlátok.

A jogosultságkezelésnek kliensoldalon és Firestore rules szinten is meg kell jelennie, mert a kliensoldali elrejtés onmagában nem elég.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát merlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

A tiltott felhasználó kezelése fontos üzleti eset, mert egy nem fizető vagy problematikus vásárló nem kaphat korlátlan hozzáférést.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát merlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

A kuponoknál a visszaélés lehetőségét úgy kell vizsgálni, hogy ki hozhat létre kupont, ki használhatja, és hogyan ellenőrizhető a kedvezmény.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

A PDF dokumentum személyes adatokat tartalmazhat, ezért a letöltés és elérés kérdésével adatvédelmi szempontból is meg kell említeni.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

Az AI proxy CORS beállítása hasznos, de önmagában nem teljes védelmi rendszer, ezért rate limit vagy kvóta továbbfejlesztésként szerepelhet.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

Tesztelés és validáció

A tesztelési fejezetnek ki kell mutatnia, hogy a kritikus folyamatokat nem csak egyszer kipróbáltad, hanem módszertan szerint ellenőrizted.

A build sikeressége azt bizonyítja, hogy az alkalmazás fordítása hibátlanul lefut egy tiszta állapotban.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDL Webshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell huzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyítsd. Ha egy funkció működését állítod, akkor mellette jeleníts meg képernyőképet, teszteredményt, adatmodell-részletet vagy rövid kód-részletet is.

Az automata tesztek a kód bizonyos logikai részeit ellenőrzik, például kosár, kupon, admin státuszváltás vagy validáció.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDL Webshop példával alátámasztani. Például egy olyan képernyőképpel vagy kód-részlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódrészlet is.

A kezi tesztek a teljes felhasználói utat fedik le, ideértve a UI állapotokat és a szerepkörök közötti különbséget.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódrészlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódrészlet is.

A GitHub Actions zöld futása külön bizonyíték arra, hogy a projekt nem csak lokálisan működött.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódrészlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódrészlet is.

A tesztelés végeén érdemes maradék kockázatokat is felsorolni, mert ettől a fejezet mérnökiileg hitelesebb lesz.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódrészlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentosege, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Erdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódrészlet is.

Reprodukálhatóság és repo-higiénia bővítés

A reprodukálhatóság azt jelenti, hogy a bíráló vagy témavezető a repóból meg tudja érteni és el tudja indítani a rendszert.

A README-ben szerepelnie kell a telepítésnek, indításnak, buildnek, teszteknek és a szükséges környezeti változóknak.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A `.env.example` nem titkok tárolására van, hanem arra, hogy megmutassa, milyen konfigurációs kulcsok szükségesek.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A `node_modules` és `build` mappák kizárása azért fontos, mert ezek generált vagy gépfüggő állományok.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A dokumentációs fájloknak a repóban a projektet kell alátámasztaniuk, a személyes segédanyagokat külön kell kezelni.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bírálo számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A CI képernyőkép és a teszteredmény a dolgozatban azt bizonyítja, hogy a kód aktuális állapota ellenőrizhető volt.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

MI-használat bővítés

Az MI-használati fejezetnek szintén kell bemutatnia, hol segített AI, és hol maradt a döntés, ellenőrzés és felelősség a fejlesztőnél.

Az AI a fejlesztés során ötletekben, hibakeresésben, dokumentációs vázlatokban és ellenőrző listákban segített.

A fejezet vegleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A vegleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-reszlet vagy rövid kódreszlet is.

A kód és dokumentáció nem tekinthető automatikusan helyesnek csak azért, mert AI javasolta; minden kimenetet futtatással vagy saját ellenőrzéssel kellett validálni.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

A dolgozatban külön kell említeni, hogy a webshopba beépített AI asszisztens más kategória, mint a fejlesztést segítő AI eszközök.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

Az AI asszisztensnel fontos korlat, hogy nem adhat korlatlan vagy ellenőrizetlen termékajánlást, hanem a katalogushoz és domainhez kötött válaszokat kell adnia.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

A végleges szövegben kerülni kell a marketinges általánosságokat; konkrét példákat érdemes írni arra, hol gyorsított és hol kellett javítani az AI javaslatát.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDLWebshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszteredmény, adatmodell-részlet vagy rövid kódreszlet is.

Osszegzes es tovaabfejlesztés bővítés

Az összegzés akkor erős, ha nem csak megismétli a fejezeteket, hanem saját értékelést is ad a projektről.

A projekt fő eredménye egy olyan webshop MVP, amely a vásárlói és admin oldali folyamatokat egy rendszerben mutatja be.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDL Webshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolódó ábra vagy táblázat ne díszítés legyen, hanem bizonyíték. Ha egy funkció működését állítod, akkor mellette jelenjen meg képernyőkép, teszt eredmény, adatmodell-részlet vagy rövid kódreszlet is.

A legfontosabb mernoki eredmények között szerepel a jogosultsági modell, Firestore rules, CSV import, PDF bizonylat, AI proxy és tesztelhető workflow.

A fejezet végleges változatában ezt a gondolatot érdemes egy konkrét TDL Webshop példával alátámasztani. Például egy olyan képernyőképpel vagy kódreszlettel, ahol látszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer működésében is megjelenik.

Szakdolgozati szempontból ennek az a jelentősége, hogy a rendszer nem különálló funkciók listája, hanem összefüggő folyamatokból álló alkalmazás. A bíráló számára ez akkor lesz egyértelmű, ha a szövegben a cél, a megvalósítás és a teszt bizonyítéka egymásra hivatkozik.

A végleges szövegben ezt a részt saját nyelvezetre kell húzni. Érdemes beleírni, hogy a fejlesztés közben milyen döntéshelyzet alakult ki, milyen alternatívát mérlegeltél, és miért a jelenlegi megoldás mellett döntöttél.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A korlatok kozott erdemes megemliteni az eles fizetesi integracio hianyát, a webes rendeles szerveroldali ujraszamitasanak tovabbfejleszteset es az AI rate limitet.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A saját reflexio részben leirható, hogy a projekt során a legtöbb munka nem az első latványos felület, hanem a hibák, edge case-ek és dokumentálás körül jelent meg.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezesi dontes nem csak elmeleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciok listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a reszt saját nyelvezetre kell huzni. Erdemes beleirni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegetelt, és miért a jelenlegi megoldás mellett döntött.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

A tovaabfejlesztési irányok legyenek realisak: online fizetes, szamlazo integracio, fejlettebb keszletstatistika, admin riportok, pontosabb AI tudastar.

A fejezet vegleges valtozataban ezt a gondolatot erdemes egy konkret TDLWebshop peldaval alatasztani. Peldaul egy olyan kepernyokeppel vagy kodreszlettel, ahol latszik, hogy a tervezési döntés nem csak elméleti, hanem a rendszer mukodeseben is megjelenik.

Szakdolgozati szempontbol ennek az a jelentosege, hogy a rendszer nem kulonallo funkciók listaja, hanem osszefuggo folyamatokbol allo alkalmazas. A biralo számára ez akkor lesz egyertelmu, ha a szovegben a cel, a megvalositas es a teszt bizonyiteka egymásra hivatkozik.

A vegleges szovegben ezt a részt saját nyelvezetre kell huzni. Erdemes beleírni, hogy a fejlesztés közben milyen dönteshelyzet alakult ki, milyen alternatívát merlegeltel, és miért a jelenlegi megoldás mellett döntöttel.

A kapcsolodo abra vagy tablazat ne diszites legyen, hanem bizonyitek. Ha egy funkcio mukodeset allitod, akkor mellette jelenjen meg kepernyokep, teszteredmeny, adatmodell-reszlet vagy rovid kodreszlet is.

Kiemelt bizonyitekok osszefoglalo tablazata

A dolgozatba javasolt legfontosabb bizonyitekok

Bizonyitek	Hova keruljon	Mit igazol
GitHub Actions zold CI kepernyokep	Reprodukalhatóság és teszteles fejezet	A projekt tiszta környezetben is buildelhető és tesztelhető.
Firestore rules kodreszlet	Biztonsági fejezet	A jogosultságok nem csak kliensoldalon, hanem adatbázis szinten is kezeltek.
Checkout validacio kepernyokep	GUI/UX és teszteles fejezet	A hibás bemeneteket a rendszer felhasználói szinten is kezeli.
Helyszini vasarlas kepernyokep	Megvalositas fejezet	Az admin oldali belso bolti folyamat is mukodik.
PDF bizonylat kepernyokep	Megvalositas és eredmények	A rendelési adatból strukturált

	fejezet	dokumentum készul.
AI asszisztens képernyőképek	MI-használat és megvalósítás fejezet	A rendszerben működő AI funkció elkülönül a fejlesztést segítő AI-használattól.

11. Reprodukálhatóság, CI és deploy

A konzulensi visszajelzés alapján a reprodukálhatóság kulcsfontosságú. Egy szakdolgozati repo akkor bírható nyugodtan, ha egy másik környezetben is elindítható, és a szükséges lépések nem csak a fejlesztő saját gépen ismertek. Emiatt a README, .env.example, telepítési leírás, demo szerepkörök és CI bizonyíték kiemelt szerepet kapnak.

A repo nem tartalmazhat node_modules, build mappákat, valódi jelszavakat, API kulcsokat vagy gépfüggő állományokat. A függősegek package.json és lock file alapján telepíthetők, a titkok pedig külön környezeti változókban vagy szolgáltatói secretként tarolandók.

A deploy két külön részből állhat: a frontend Firebase Hostingra kerül, az AI asszisztens szerveroldali proxyja pedig Cloudflare Workerként működik. A dolgozatban ezt a szétválasztást érdemes röviden elmagyarázni, mert jól mutatja a kulcsvédelem és a kliens/szerver felelősségi határainak megértését.

11. táblázat - Reprodukciós és deploy lépések

Lépés	Parancs / ellenőrzés	Cél
Függősegek telepítése	npm install	Angular projekt függősegeinek telepítése.
Build	npm run build	Frontend fordíthatóságának ellenőrzése.
Teszt	npm test -- --watch=false	Automata tesztek futtatása.
Hosting deploy	firebase deploy --only hosting	Frontend publikálása.
Firestore rules deploy	firebase deploy --only firestore:rules	Biztonsági szabályok elcsatása.
Worker deploy	npx wrangler deploy	OpenRouter proxy publikálása.

12. Mesterseges intelligencia hasznalata a fejlesztés során

A szakdolgozat készítése során mesterséges intelligencia több ponton segítette a munkát. Fontos különválasztani két területet: az egyik a fejlesztési folyamatban használt AI-támogatás, a másik pedig maga a TDLWebshopban megjelenő vásárlói AI asszisztens. A kettő nem ugyanaz: az előbbi fejlesztői segédeszköz, az utóbbi a rendszer egyik funkciója.

A fejlesztés során AI-támogatás elsősorban ötletelesre, kódreview-ra, hibakeresésre, dokumentációs szerkezet kialakítására, tesztforgatókönyvek összeállítására és nyelvi javításokra lett használva. A kimenetek nem automatikusan kerültek elfogadásra: a fejlesztő feladata maradt a kód megértése, ellenőrzése, futtatása és a döntések meghozatala.

Az AI használata során külön figyelmet kellett fordítani a titkok kezelésére. API kulcsot vagy jelszót nem szabad a kódba vagy nyilvános repóba helyezni. Egy korábbi kulcsot vissza kellett vonni, majd a véglegesebb megoldásban az OpenRouter kulcs Cloudflare Worker secretként került kezelésre.

Az AI korlátai a fejlesztésben is megjelentek. Előfordulhat, hogy egy javaslat elavult, nem illeszkedik a projekt szerkezetébe, vagy túl általános megfogalmazást ad. Emiatt a végleges szakdolgozati szöveget saját nyelvezetre kell átírni, és minden állítást a működő rendszerrel, tesztekkel vagy dokumentációval kell alátámasztani.

A rendszerben működő AI asszisztens szinten korlátos. Nem célja, hogy szakmERNOKI felelősséggel teljes műszaki ajánlatot adjon. A cél inkább az, hogy a termékkatalógus alapján irányt mutasson, és ha nincs pontos találat vagy biztos adat, akkor javasolja az emailt vagy személyes egyeztetést.

12. táblázat - AI-használat és ellenőrzés

AI-használat területe	Mire szolgált	Validáció
Ötleteles	Funkciók, MVP-határ és dokumentációs fejezetek átgondolása.	Konzulensi elvárásokkal összevetve.
Kódreview	Hibalehetőségek, jogosultságok, validáció és tranzakciók vizsgálata.	Kód olvasása, build és teszt.
Dokumentáció	Fejezetváz, tesztjegyzék és biztonsági minimum	Saját átirás és szakmai ellenőrzés.

	elokeszítése.	
AI asszisztens funkció	Felhasználói kérdések katalógushoz kötött megválaszolása.	Domainkérdés, termékkérdés és irreleváns kérdés tesztje.

13. Összefoglalás, korlatok és továbbfejlesztés

A TDLWebshop egy épütelgépészeti témájú webshop és adminisztrációs rendszer MVP-je. A projekt bemutatja, hogyan épülhet fel egy vásárlói oldalból, admin felületből, Firestore adatmodellből, jogosultsági szabályokból, PDF bizonylatból, CSV importból, készletkezelésből és AI asszisztensből álló webes rendszer.

A munka legerősebb része a funkciók összekapcsolása: a vásárlói rendelés nem önmagában áll, hanem megjelenik az admin felületen, státuszt kap, PDF bizonylattal és készletlogikával kapcsolódik a belső folyamatokhoz. A helyszíni vásárlás és mentett vásárló kezelése külön domaintereket ad a rendszernek.

A rendszernek vannak tudatos korlátai. Ilyen a teljes pénzügyi integráció hiánya, a webes rendelések szerveroldali arújrászmólasának továbbfejlesztési igénye, a készletfoglalás ipari szintű kezelése és az AI asszisztens rate limit/kvóta oldali erősítésének szükségessége. Ezeket nem hiányosságként, hanem továbbfejlesztési irányként kell bemutatni.

[SAJÁT ZÁRO REFLEXIO HELYE] Ide 1-2 bekezdésben saját hangon írd le, mit tanultál a projektből, mi volt a legnehezebb, mit csinálnál másként, és melyik részere vagy a legbuszkebb. Ez legyen teljesen saját megfogalmazás, mert a védesen is ebből tudsz a legtermészetesebben beszélni.

Irodalomjegyzék

Angular dokumentáció - <https://angular.dev/>

Firebase dokumentáció - <https://firebase.google.com/docs>

Cloud Firestore Security Rules dokumentáció - <https://firebase.google.com/docs/firestore/security/get-started>

Cloudflare Workers dokumentáció - <https://developers.cloudflare.com/workers/>

OpenRouter dokumentáció - <https://openrouter.ai/docs>

GitHub Actions dokumentacio - <https://docs.github.com/actions>

[További piaci összehasonlításban használt webshopok forrásai]

[Intézményi szakdolgozati formai és tartalmi útmutató]

Mellekletek

M1. Kezi tesztjegyzék könyv kitöltött állapotban.

M2. Kiemelt kódrészletek: checkout, OrderService, Firestore rules, InvoiceService, AI proxy.

M3. Képernyőképek gyűjteménye: vásárlói oldal, admin oldal, AI asszisztens, CI.

M4. Reprodukciós lépések és környezeti változók mintája.